LAB-20 APP DEPLOYMENT USING ECS AND ECR

Name:- Vedant Khanna

SapID:- 5001118950

R.No:- R2142230790

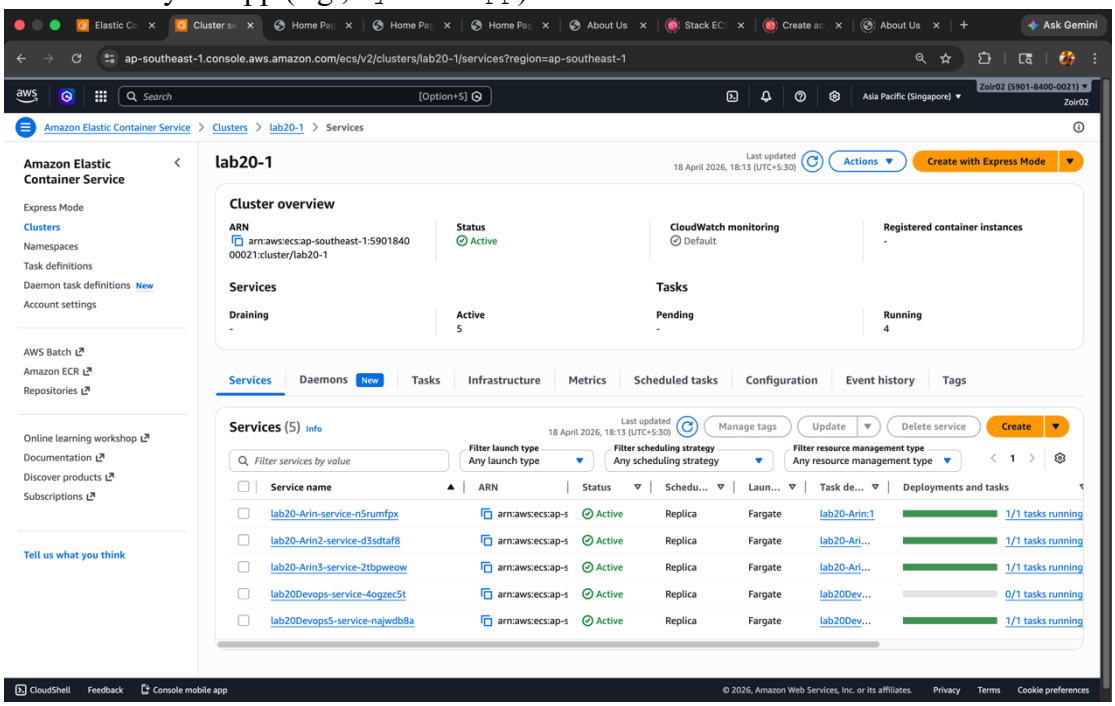
Batch:- DevOps - B1

To:- Hitesh Sharma Sir

Phase 1: Prepare and Push to Amazon ECR

ECR is your private library for Docker images.

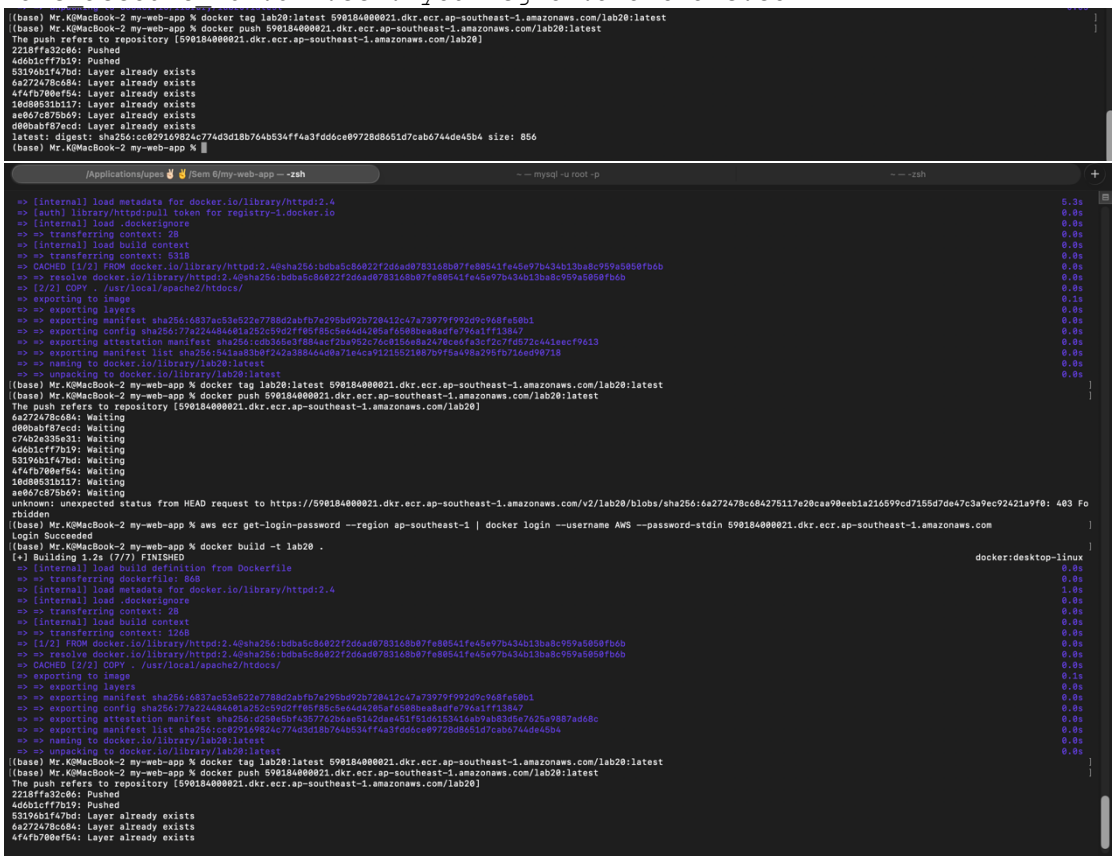
Create an ECR Repository: In the AWS Console (or via CLI), create a repository named after your app (e.g., `my-web-app`).



Authenticate Docker to ECR: Run the login command provided by the AWS console (it usually looks like this):

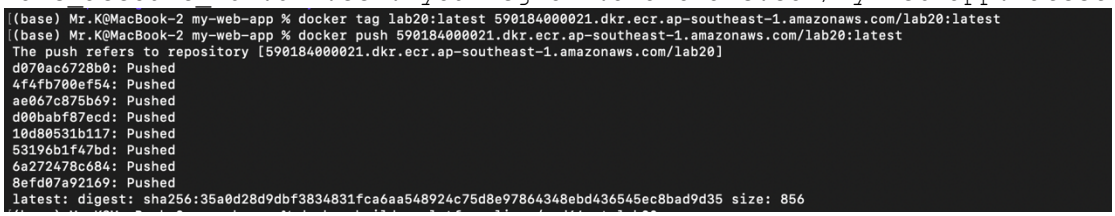
Bash

```
aws ecr get-login-password --region <your-region> | docker login -
username AWS --password-stdin
<aws_account_id>.dkr.ecr.<yourregion.amazonaws.com
```



Build and Tag your Image: Build your local Docker image and tag it with the ECR repository URI. Bash `docker build -t my-web-app .`

```
docker tag my-web-app:latest
<aws_account_id>.dkr.ecr.<yourregion>.amazonaws.com/my-web-app:latest
```



Push the Image:

Bash

```
docker push <aws_account_id>.dkr.ecr.<your-
region>.amazonaws.com/myweb-app:latest
```

```
(base) Mr.K@MacBook-2 my-web-app % docker tag lab20:latest 590184000021.dkr.ecr.ap-southeast-1.amazonaws.com/lab20:latest
(base) Mr.K@MacBook-2 my-web-app % docker push 590184000021.dkr.ecr.ap-southeast-1.amazonaws.com/lab20:latest
The push refers to repository [590184000021.dkr.ecr.ap-southeast-1.amazonaws.com/lab20]
d070ac6728b0: Pushed
4f4fb700ef54: Pushed
ae067c875b69: Pushed
d00babf87ecd: Pushed
10d80531b117: Pushed
53196b1f47bd: Pushed
6a272478c684: Pushed
8efd07a92169: Pushed
latest: digest: sha256:35a0d28d9dbf3834831fca6aa548924c75d8e97864348ebd436545ec8bad9d35 size: 856
```

Phase 2: Set up Amazon ECS

ECS manages where and how your containers run.

1. Create a Cluster

A cluster is a logical grouping of tasks or services. In the ECS console, choose **Create Cluster**. For most modern apps, select **Fargate (Serverless)**—this means you don't have to manage underlying EC2 servers.

2. Create a Task Definition

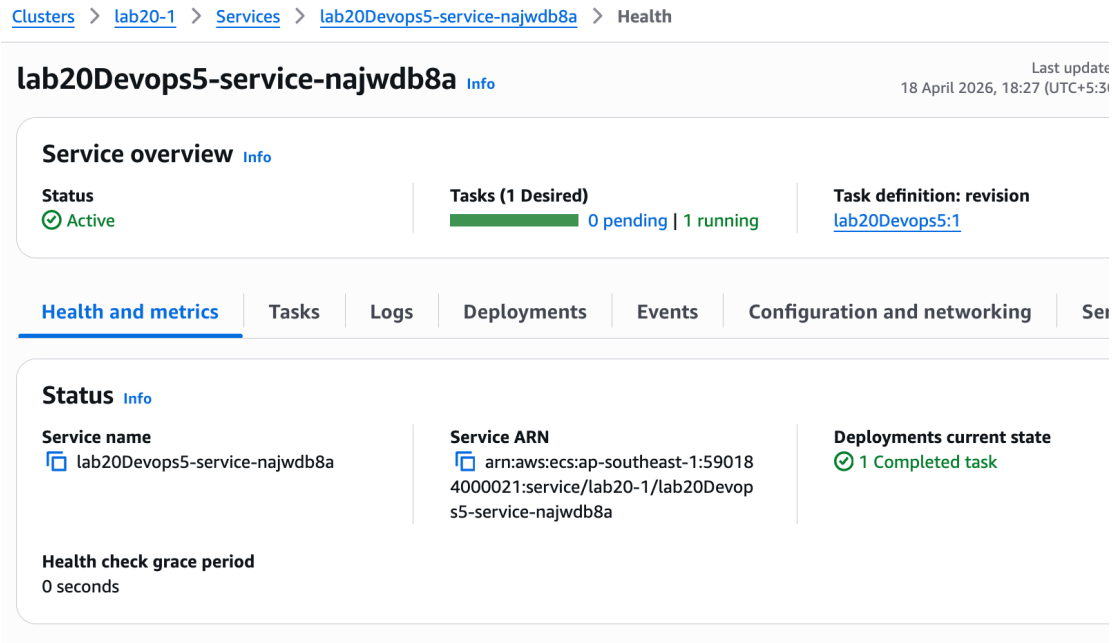
Think of the **Task Definition** as the "blueprint" for your app.

Launch Type: Fargate.

Container Image: Paste the ECR Image URI you pushed in Phase 1.

Port Mappings: Usually port `80` or `8080` (whatever your app listens on).

Resources: Assign CPU and Memory (e.g., 0.5 vCPU and 1 GB RAM).



3. Create a Service

A **Service** tells ECS how many copies of your Task Blueprint to keep running.

Desired Tasks: Set this to `2` or more for high availability.

Load Balancing: (Recommended) Select an **Application Load Balancer (ALB)**. This provides a single URL to access your app and distributes traffic between your running containers.

Networking: Ensure your VPC and Security Groups allow traffic on your application's port.

Phase 3: Access and Update

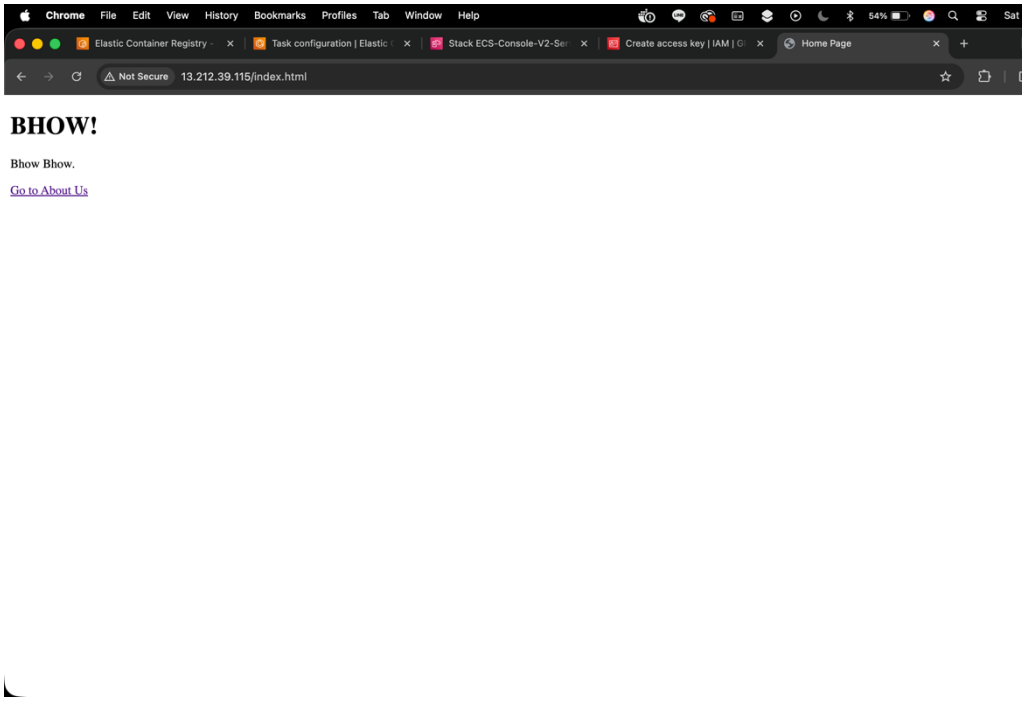
Verification: Once the service status says "Active" and the tasks are "Running," go to the **Load Balancing** section to find your **DNS Name**. Paste that into your browser to see your app.

Updating: To deploy a new version, simply push a new image to ECR with the same tag (or a new one), update the Task Definition with the new image URI, and update the Service to use the latest revision. ECS will automatically perform a **Rolling Update**, replacing old containers with new ones without downtime.

Quick Comparison: EC2 vs. Fargate

Feature	EC2 Launch Type	Fargate Launch Type
Management	You manage the servers	AWS manages the servers (Serverless)

Feature	EC2 Launch T	Fargate Launch Type
Cost	Pay for the inst	Pay for CPU/RAM per second
Complex	High (patching,	Low (just provide the container)



About Us

~Mr.K from devops ðŸ‘ ðŸ »,

Go back to Home